Lab 5 — Quản lý sinh viên bằng List

1. Đề bài tiếng Anh

LAB Session 5

Objective

Develop a console-based Student Management System in Python using a **list** as the main data structure.

Requirements

Write a Python program that implements the following features:

1. **Add a student:** Input student information (ID, name, score). Store the student in a list.
2. **Display student list:** Show all students in the list.
3. **Search student by name:** Input a name. Display matching students.
4. **Delete a student:** Input student ID. Remove the student from the list.
5. **Sort students by score:** Sort the list in ascending or descending order based on score.
6. **Statistics:** Find the student with the highest score; find the student with the lowest score; calculate the average score.
7. **Exit program.**

Đề được chuyển từ `Lab Session 5.docx` sang Markdown.

2. Đề bài tiếng Việt

LAB SESSION 5 — Quản lý sinh viên bằng List (danh sách)

Mục tiêu

Xây dựng hệ thống quản lý sinh viên trên console bằng Python, sử dụng **list** làm cấu trúc lưu trữ chính.

Yêu cầu

Viết chương trình Python có các chức năng:

1. **Thêm sinh viên:** Nhập ID, tên, điểm; lưu vào một danh sách.
2. **Hiển thị danh sách sinh viên:** In toàn bộ sinh viên trong list (danh sách).
3. **Tìm sinh viên theo tên:** Nhập tên, hiển thị sinh viên khớp.
4. **Xóa sinh viên:** Nhập ID, xóa sinh viên tương ứng khỏi danh sách.
5. **Sắp xếp theo điểm:** Sắp xếp danh sách tăng dần hoặc giảm dần theo điểm.
6. **Thống kê:** Tìm sinh viên có điểm cao nhất, thấp nhất và tính điểm trung bình.
7. **Thoát chương trình:** Cho phép người dùng kết thúc menu.

3. Code Python có chú thích

```
import sys  # Nạp thư viện (import: nạp vào) để sử dụng hàm sẵn có.

def validate(student_id,name,score):  # Khai báo hàm (function: hàm) để gom và gọi lại những bước này.
    sid=student_id.strip()  # Bỏ khoảng trắng đầu/cuối bằng strip (cắt khoảng trắng).
    name=name.strip()  # Bỏ khoảng trắng đầu/cuối bằng strip (cắt khoảng trắng).
    if not sid or not name: raise ValueError("ID and name are required")  # Kiểm tra điều kiện; nhánh sau chỉ chạy khi điều kiện đúng.
    try: score=float(score)  # Thử thao tác có thể gây lỗi và chuẩn bị xử lý lỗi.
    except ValueError as e: raise ValueError("Score must be numeric") from e  # Bắt lỗi ngoại lệ (exception: lỗi) để báo cho người dùng.
    if not 0<=score<=10: raise ValueError("Score must be in 0..10")  # Kiểm tra điều kiện; nhánh sau chỉ chạy khi điều kiện đúng.
    return {"id":sid,"name":name,"score":score}  # Trả kết quả (return: trả về) cho nơi gọi hàm.

def add(students,student_id,name,score):  # Khai báo hàm (function: hàm) để gom và gọi lại những bước này.
    data=validate(student_id,name,score)  # Tính hoặc cập nhật biến data theo công thức của bài.
    if any(item["id"]==data["id"] for item in students):  # Kiểm tra điều kiện; nhánh sau chỉ chạy khi điều kiện đúng.
        raise ValueError("Duplicate student ID")  # Chủ động báo lỗi đầu vào thay vì chấp nhận dữ liệu sai.
    students.append(data)  # Thêm phần tử vào cuối danh sách bằng append (thêm cuối).

def find_by_name(students,name):  # Khai báo hàm (function: hàm) để gom và gọi lại những bước này.
    query=name.strip().casefold()  # Bỏ khoảng trắng đầu/cuối bằng strip (cắt khoảng trắng).
    if not query: return []  # Kiểm tra điều kiện; nhánh sau chỉ chạy khi điều kiện đúng.
    return [s for s in students if query in s["name"].casefold()]  # Trả kết quả (return: trả về) cho nơi gọi hàm.

def delete_by_id(students,student_id):  # Khai báo hàm (function: hàm) để gom và gọi lại những bước này.
    for index,s in enumerate(students):  # Duyệt tuần tự từng phần tử bằng vòng lặp (loop: lặp).
        if s["id"]==student_id:  # Kiểm tra điều kiện; nhánh sau chỉ chạy khi điều kiện đúng.
            return students.pop(index)  # Trả kết quả (return: trả về) cho nơi gọi hàm.
    raise KeyError(student_id)  # Chủ động báo lỗi đầu vào thay vì chấp nhận dữ liệu sai.

def sorted_by_score(students,descending=False):  # Khai báo hàm (function: hàm) để gom và gọi lại những bước này.
    return sorted(students,key=lambda s:s["score"],reverse=descending)  # Trả kết quả (return: trả về) cho nơi gọi hàm.

def statistics(students):  # Khai báo hàm (function: hàm) để gom và gọi lại những bước này.
    if not students: return None  # Kiểm tra điều kiện; nhánh sau chỉ chạy khi điều kiện đúng.
    return {"highest":max(students,key=lambda s:s["score"]),  # Trả kết quả (return: trả về) cho nơi gọi hàm.
            "lowest":min(students,key=lambda s:s["score"]),  # Gắn tên trường (key: khóa) với giá trị trong từ điển.
            "average":sum(s["score"] for s in students)/len(students)}  # Gắn tên trường (key: khóa) với giá trị trong từ điển.

def show(students):  # Khai báo hàm (function: hàm) để gom và gọi lại những bước này.
    for s in students:  # Duyệt tuần tự từng phần tử bằng vòng lặp (loop: lặp).
        print(f'{s["id"]} | {s["name"]} | {s["score"]:g}/10')  # In kết quả (output: kết quả) để người học đối chiếu.

def sample():  # Khai báo hàm (function: hàm) để gom và gọi lại những bước này.
    students=[]  # Tính hoặc cập nhật biến students theo công thức của bài.
    grades=[5,6,7,8,9,10,5.5,6.5,7.5,8.5,9.5,4]  # Tính hoặc cập nhật biến grades theo công thức của bài.
    for i,score in enumerate(grades,1):  # Duyệt tuần tự từng phần tử bằng vòng lặp (loop: lặp).
        add(students,f"S{i:03}",f"Learner {i:02}",score)  # Thực hiện bước xử lý, giữ nguyên cấu trúc và thứ tự code mẫu.
    return students  # Trả kết quả (return: trả về) cho nơi gọi hàm.

def run_demo():  # Khai báo hàm (function: hàm) để gom và gọi lại những bước này.
    students=sample()  # Tính hoặc cập nhật biến students theo công thức của bài.
    if not (len(students)==12): raise ValueError("Kiểm tra dữ liệu không đạt")  # Kiểm tra bằng if (nếu); khi điều kiện sai sẽ báo lỗi rõ ràng.
    add(students,"S013","Learner 13",7)  # Thực hiện bước xử lý, giữ nguyên cấu trúc và thứ tự code mẫu.
    if not (len(find_by_name(students,"learner"))==13): raise ValueError("Kiểm tra dữ liệu không đạt")  # Kiểm tra bằng if (nếu); khi điều kiện sai sẽ báo lỗi rõ ràng.
    removed=delete_by_id(students,"S013")  # Tính hoặc cập nhật biến removed theo công thức của bài.
    if not (removed["id"]=="S013" and len(students)==12): raise ValueError("Kiểm tra dữ liệu không đạt")  # Kiểm tra bằng if (nếu); khi điều kiện sai sẽ báo lỗi rõ ràng.
```

```
    print("Sorted:",[(s["id"],s["score"]) for s in sorted_by_score(students)])  # In kết quả (output: kết quả) để người học đối chiếu.
    print("Statistics:",statistics(students))  # In kết quả (output: kết quả) để người học đối chiếu.
    show(students)  # Thực hiện bước xử lý, giữ nguyên cấu trúc và thứ tự code mẫu.

def interactive():  # Khai báo hàm (function: hàm) để gom và gọi lại những bước này.
    students=sample()  # Tính hoặc cập nhật biến students theo công thức của bài.
    while True:  # Tiếp tục vòng lặp khi điều kiện còn đúng.
        print("\n1.Add 2.Display 3.Search 4.Delete 5.Sort 6.Statistics 0.Exit")  # In kết quả (output: kết quả) để người học đối chiếu.
        c=input("Choice: ").strip()  # Bỏ khoảng trắng đầu/cuối bằng strip (cắt khoảng trắng).
        try:  # Thử thao tác có thể gây lỗi và chuẩn bị xử lý lỗi.
            if c=="0":break  # Kiểm tra điều kiện; nhánh sau chỉ chạy khi điều kiện đúng.
            elif c=="1": add(students,input("ID: "),input("Name: "),input("Score: "))  # Xét điều kiện tiếp theo nếu nhánh trước không thỏa.
            elif c=="2":show(students)  # Xét điều kiện tiếp theo nếu nhánh trước không thỏa.
            elif c=="3":show(find_by_name(students,input("Name to search: ")))  # Xét điều kiện tiếp theo nếu nhánh trước không thỏa.
            elif c=="4":delete_by_id(students,input("ID: ").strip())  # Xét điều kiện tiếp theo nếu nhánh trước không thỏa.
            elif c=="5":  # Xét điều kiện tiếp theo nếu nhánh trước không thỏa.
                show(sorted_by_score(students,input("Descending? [y/N] ").lower()=="y"))  # Nhận input (dữ liệu nhập) để xử lý và kiểm tra.
            elif c=="6":print(statistics(students))  # Xét điều kiện tiếp theo nếu nhánh trước không thỏa.
            else:print("Unknown menu option")  # Xử lý trường hợp ngược lại hoặc còn lại.
        except (ValueError,KeyError) as exc:  # Bắt lỗi ngoại lệ (exception: lỗi) để báo cho người dùng.
            print("Error:",exc)  # In kết quả (output: kết quả) để người học đối chiếu.
if __name__=="__main__":  # Kiểm tra điều kiện; nhánh sau chỉ chạy khi điều kiện đúng.
    run_demo() if "--demo" in sys.argv else interactive()  # Thực hiện bước xử lý, giữ nguyên cấu trúc và thứ tự code mẫu.
```

4. Giải thích nội dung và cách giải

Câu 1

Đầu tiên dùng `validate()` từ code nguồn để làm sạch ID/tên và đổi điểm về `float`. Sau đó kiểm tra trùng ID bằng `any` và dùng `append` thêm bản ghi vào list (danh sách). Các ví dụ DEMO ban đầu đưa thông tin sinh viên vào list nhưng chưa thống nhất kiểu dữ liệu, nên bản tham chiếu kiểm tra kỹ hơn.

Vì sao dùng list? Đề yêu cầu lưu sinh viên vào danh sách; mỗi phần tử có ID, tên và điểm, nên một dictionary cho từng người giúp đọc theo tên trường. **Quy trình:** Nhận dữ liệu → bỏ khoảng trắng → đổi điểm thành `float` → kiểm tra điểm trong 0–10 theo quy ước code → tạo dictionary và `append` vào danh sách.

```
student = {"id": "S01", "name": "An", "score": 8.5}
if 0 <= student["score"] <= 10:
    students.append(student)
```

Giải thích: `{"id": ...}` lưu theo cặp khóa/giá trị; `.append(...)` thêm vào cuối list; `0 <= score <= 10` kiểm tra gồm cả hai biên. Mốc điểm này là ràng buộc của code tham khảo, không phải dữ liệu được ghi rõ trong đề Word.

Điểm cần kiểm tra: list là cấu trúc chứa chính; mỗi phần tử dùng dictionary (từ điển) có ID/tên/điểm.

Ví dụ và lỗi thường gặp: List (danh sách) chứa các bản ghi có mã, tên, điểm. Dùng append() nối bản ghi mới và kiểm tra mã trùng trước khi thêm.

Thuật ngữ cần nhớ: List (danh sách).

Câu 2

Dùng vòng lặp duyệt từng phần tử của list (danh sách) và truy cập các khóa `id`, `name`, `score` của từng phần tử. Không dùng dictionary (từ điển) làm cấu trúc chứa danh sách chính.

Vì sao phải duyệt? List chứa nhiều dictionary, nên cần lấy từng bản ghi mới hiển thị đầy đủ. **Quy trình:** Xác định danh sách có rỗng không → duyệt theo thứ tự đã lưu → in ID, tên, điểm → đối chiếu số bản ghi.

```
total = len(students)
for student in students:
    print(student["id"], student["name"], student["score"])
```

Giải thích: `len` đếm người, không tính điểm; `for ... in` lấy mỗi dictionary trong list; `student["name"]` là truy cập trường tên. List rỗng khiến vòng lặp không in hàng nào, cần thông báo phù hợp trong chương trình.

Điểm cần kiểm tra: thêm qua `append` và kiểm tra trùng mã nếu code tham khảo có hỗ trợ.

Ví dụ và lỗi thường gặp: Duyệt từng bản ghi trong list (danh sách) và in đủ trường. Nếu rỗng, cần hiển thị thông báo phù hợp.

Thuật ngữ cần nhớ: Display (hiển thị).

Câu 3

Chuẩn hóa tên bằng `.strip().casefold()` rồi lọc phần tử chứa chuỗi tìm kiếm. Khác với DEMO so sánh tên chính xác, cách này thuận tiện với dữ liệu thực tế.

Vì sao tìm không phân biệt chữ hoa? Người dùng có thể nhập tên khác kiểu viết; `casefold` chuẩn hóa chữ khi so sánh, giúp tìm một phần tên thay vì phải trùng nguyên văn. **Quy trình:** Nhận từ khóa → xóa khoảng trắng → chuẩn hóa chữ → duyệt sinh viên → lấy bản ghi có tên chứa từ khóa.

```
query = "an".strip().casefold()
matches = [s for s in students if query in s["name"].casefold()]
```

Giải thích: `.strip()` bỏ khoảng trắng hai đầu; `.casefold()` chuẩn hóa chữ mạnh hơn `.lower()` cho so sánh không phân biệt hoa/thường; `in` kiểm tra chuỗi con; danh sách kết quả vẫn có thể rỗng. Không nhầm `in` với so khớp ID chính xác.

Điểm cần kiểm tra: tìm tên bằng `casefold()`; duyệt và xóa theo ID bằng `enumerate()/pop()`.

Ví dụ và lỗi thường gặp: Tìm bằng casefold() để không phân biệt hoa/thường; dùng chuỗi con nếu code tham khảo có hỗ trợ.

Thuật ngữ cần nhớ: Search (tìm kiếm).

Câu 4

Dùng `enumerate` lấy cặp chỉ số và bản ghi, sau đó `pop(index)` xóa đúng sinh viên có mã cần tìm. Nếu không thấy ID, báo lỗi.

Vì sao xóa theo ID? Tên có thể trùng, còn mã sinh viên được dùng để xác định một người. Phải tìm đúng mã rồi mới xóa. **Quy trình:** Nhận ID → duyệt danh sách đến dòng khớp → xóa dòng đã xác định → dừng vòng lặp → báo không tìm thấy nếu cần.

```
student_id = "S01"
index = next((i for i, s in enumerate(students) if s["id"] == student_id), None)
if index is not None:
    students.pop(index)
```

Giải thích: `enumerate` trả cặp vị trí và phần tử; `next(..., None)` lấy khớp đầu hoặc `None`; `pop(index)` xóa theo vị trí; `is not None` phân biệt vị trí 0 với không tìm thấy. Việc duyệt có thể cần xem đến `n` người (O(n)).

Điểm cần kiểm tra: sắp xếp bằng `sorted(..., key=...)`; không so sánh điểm như chuỗi.

Ví dụ và lỗi thường gặp: Dùng enumerate() tìm đúng chỉ số và pop() xóa mục được chọn; ID không tồn tại phải báo lỗi.

Thuật ngữ cần nhớ: Delete (xóa).

Câu 5

Dùng `sorted(students, key=..., reverse=...)` để tạo kết quả có thứ tự theo điểm số; không sắp chuỗi điểm thay cho số.

Vì sao phải chọn khóa sắp xếp? List lưu cả ID và tên; gọi `sorted` mà không chỉ rõ sẽ không biết cần xếp theo điểm. **Quy trình:** Chọn khóa điểm → dùng `sorted` tạo danh sách kết quả → dùng tùy chọn tăng/giảm theo yêu cầu → in để so sánh.

```
ascending = sorted(students, key=lambda s: s["score"], reverse=False)
descending = sorted(students, key=lambda s: s["score"], reverse=True)
```

Giải thích flag: `key=lambda s: s["score"]` lấy điểm của mỗi sinh viên làm khóa; `reverse=False` tăng dần, `reverse=True` giảm dần; `sorted` trả list **mới**, không sửa list cũ. Chi phí sắp xếp thường tăng theo khoảng `n * log(n)` chứ không phải toán tử tính điểm.

Điểm cần kiểm tra: điểm trung bình = tổng điểm/số sinh viên; xử lý list (danh sách) rỗng.

Ví dụ và lỗi thường gặp: sorted() sắp theo điểm số đã đổi thành float, không sắp theo chuỗi văn bản. reverse=True đảo chiều.

Thuật ngữ cần nhớ: Sort (sắp xếp).

Câu 6

Với list (danh sách) không rỗng, dùng `max` và `min` theo khóa điểm để tìm toàn bộ bản ghi đứng đầu/cuối, rồi cộng điểm chia số sinh viên.

Vì sao tính trên cả danh sách? Điểm cao nhất, thấp nhất và trung bình phải phản ánh đúng toàn bộ sinh viên hiện có; nếu không có ai thì không thể chia cho 0. **Quy trình:** Kiểm tra list rỗng → trích cột điểm → tìm max/min → cộng điểm và chia số người.

```
scores = [s["score"] for s in students]
if scores:
    highest = max(scores)
    lowest = min(scores)
    average = sum(scores) / len(scores)
else:
    highest = lowest = average = None
```

Giải thích: `max/min` tìm cao nhất/thấp nhất, `sum` cộng điểm, `len` đếm số người, `/` chia thực; `if scores` kiểm tra list có dữ liệu. `None` thể hiện chưa có kết quả hợp lệ, không phải điểm 0.

Điểm cần kiểm tra: menu `while True` thoát khi chọn 0.

Ví dụ và lỗi thường gặp: Trung bình = tổng điểm/số sinh viên; khi không có sinh viên, tránh phép chia cho 0.

Thuật ngữ cần nhớ: Statistics (thống kê).

Câu 7

Dùng vòng `while True` hiển thị menu, chọn tác vụ theo `if/elif`, xử lý lỗi đầu vào, và `break` khi chọn Exit. Đây là chương trình đã có, không tạo menu mới.

Vì sao Exit là một lựa chọn menu? Chương trình console chạy lặp đến khi người dùng yêu cầu dừng, không tự thoát sau một thao tác. **Quy trình:** Nhận lựa chọn → so sánh với mã thoát → dừng vòng lặp, còn lại thực hiện chức năng tương ứng.

```
choice = "0"
if choice == "0":
    print("Kết thúc")
    print("Đây là ví dụ kiểm tra lựa chọn; menu chính dùng break")
```

Giải thích: `input(...)` trả chuỗi; == so sánh chuỗi lựa chọn, `break` dùng **bên trong vòng lặp** mới dừng được vòng lặp; `while True` nghĩa là chạy liên tục cho đến khi có lệnh dừng. Không có phép tính toán học cho chức năng Exit.

Điểm cần kiểm tra: menu `while True` thoát khi chọn 0.

Ví dụ và lỗi thường gặp: Menu lặp đến khi chọn Exit. Chế độ --demo cho phép thử các chức năng mà không cần nhập thủ công.

Thuật ngữ cần nhớ: Exit (thoát).